LLM-Powered Multi-Agent Planning System

Telecom Incident Management Domain

MSc Artificial Intelligence — Module: Intelligent Agents 2026
University of Essex Online

Abdullah A. Buqshan

October 2026

OVERVIEW

System Overview & Design Goals

Design Goals

- Autonomous goal-directed behavior : agent receives a high-level incident goal and decomposes it independently
- Separation of concerns : distinct agents for Planning, Retrieval, Analysis, and Reporting
- LLM reasoning core via Hugging Face free-tier (Mistral-7B-Instruct)
- Observable, testable pipeline with structured inter-agent messaging

Domain Context

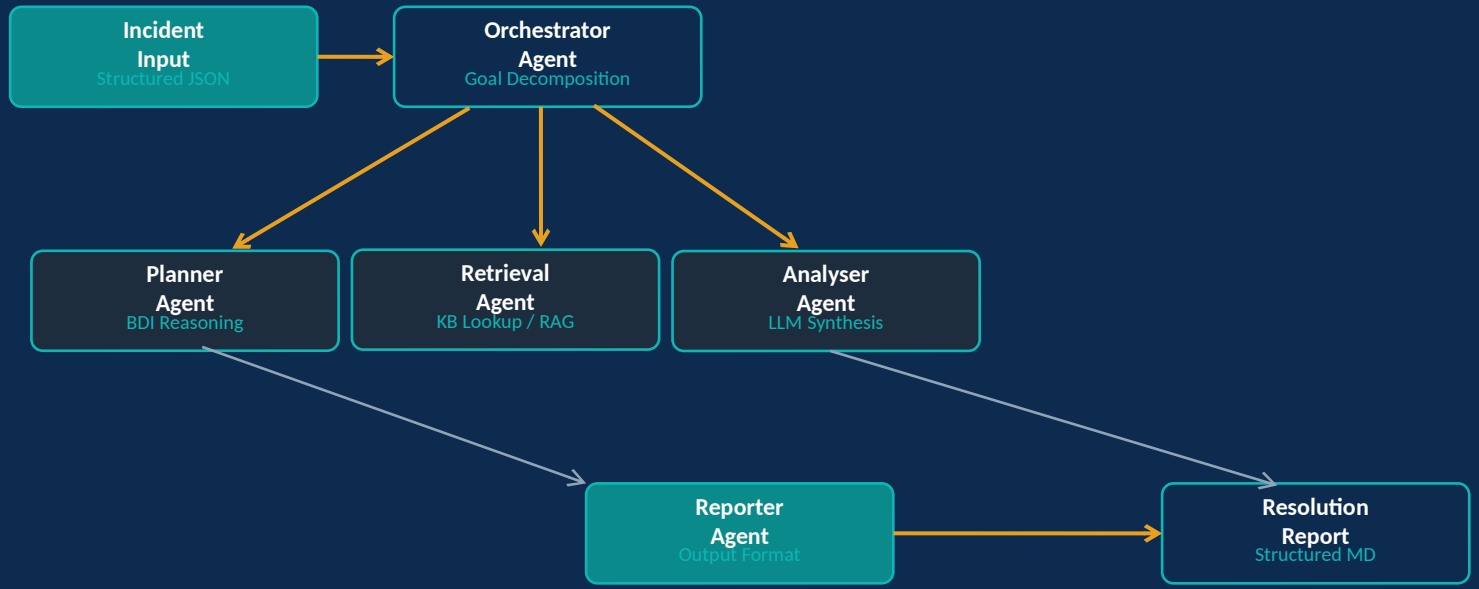
Telecom Network Operations Centre (NOC) Incident Management was selected as the application domain. This aligns with the author's 20+ years of industry experience in UAE telecommunications.

The agent system ingests a structured incident report, plans a diagnostic sequence, retrieves knowledge-base entries, synthesises a root-cause hypothesis, and generates a structured resolution recommendation.

This mirrors the BDI (Belief-Desire-Intention) model described by Wooldridge (2009), where the agent's beliefs are the incident state, its desires are restoration and root-cause identification, and its intentions are the planned action sequence.

ARCHITECTURE

Multi-Agent System Architecture



AGENT DESIGN

Agent Roles & Responsibilities

Orchestrator Agent

Responsibility: Receives the high-level incident goal, decomposes it into a DAG of sub-tasks, manages message routing between agents, and aggregates final outputs.

Why: A single orchestrator avoids distributed coordination overhead and provides a single point of observability — consistent with Russell and Norvig (2021) goal-based agent model.

Planner Agent

Responsibility: Applies a BDI-style reasoning loop to generate a stepwise diagnostic plan. Outputs a structured action sequence in JSON.

Why: Separating planning from execution allows plan validation before LLM calls are made, reducing API costs and improving predictability.

Retrieval Agent

Responsibility: Queries a local FAISS vector index of telecom fault knowledge base entries using semantic similarity search (sentence-transformers).

Why: Local vector search avoids external API dependency for retrieval, keeping the system functional in offline/restricted environments.

Analyser Agent

Responsibility: Constructs a structured prompt from planner outputs and retrieved context, calls Mistral-7B-Instruct via HuggingFace Inference API, and extracts the root-cause hypothesis.

Why: Mistral-7B-Instruct was selected over GPT-4 to satisfy the free-tier constraint while maintaining sufficient reasoning capability for structured tasks.

Reporter Agent

Responsibility: Formats agent outputs into a structured Markdown incident resolution report including root cause, confidence level, recommended actions, and escalation flag.

Why: Separating reporting from analysis enables format changes (e.g., JSON, PDF) without touching the reasoning logic.

IMPLEMENTATION

Implementation Stack & Technology Choices

Layer	Technology	Design Rationale
Language & Runtime	Python 3.11	Mature ecosystem for NLP/ML; wide library support for agent frameworks.
LLM Provider	HuggingFace Inference API (Mistral-7B-Instruct)	Free-tier; instruction-tuned; avoids OpenAI cost dependency.
Vector Retrieval	FAISS + sentence-transformers	Local, dependency-free semantic search; no external DB required.
Agent Framework	Custom orchestration (no LangChain)	Full transparency of message passing; avoids framework abstraction hiding design decisions.
Message Protocol	Structured Python dataclasses (AgentMessage)	Type-safe inter-agent communication; compatible with unit testing.
Testing	pytest + pytest-mock	Industry standard; supports mock of HuggingFace API calls for offline tests.
Knowledge Base	JSON flat-file + FAISS index	Portable, version-controllable, no external service required.
Output Format	Markdown → structured dict	Human-readable and machine-parseable; easy to extend to JSON/PDF.
Web Hosting	Streamlit Community Cloud	Free permanent public URL; deploys from GitHub; always live — no Colab session required.

CODE

Core Code: Key Design Decisions

AgentMessage Dataclass

```
@dataclass
class AgentMessage:
    sender: str
    recipient: str
    payload: dict
    timestamp: str
```

Why: Typed message contract between agents. Decouples sender/receiver — any agent can be swapped without changing the protocol.

Orchestrator Plan Loop

```
for task in plan.tasks:
    agent = self.registry[task.agent]
    msg = AgentMessage(...)
    result = agent.run(msg)
    self.log(result)
```

Why: Sequential task execution with full logging. Every step is observable — critical for debugging and for the testing requirement.

Retrieval Agent — FAISS

```
embeddings = model.encode(query)
D, I = index.search(
    embeddings, k=TOP_K)
return [kb[i] for i in I[0]]
```

Why: K-nearest-neighbour semantic search. TOP_K=3 was chosen empirically — balances context richness against prompt token budget.

Analyser — Prompt Construction

```
prompt = SYSTEM_PROMPT.format(
    incident=incident_json,
    context=retrieved_docs,
    plan_steps=plan.steps
)
```

Why: Structured prompt template separates concerns: system instructions are static, dynamic context is injected. Easier to tune and test.

EXECUTION

Execution Evidence & System Behaviour

System Log — Incident Ingestion

```
[INFO] Orchestrator received goal:
"Diagnose P1 MPLS circuit
fault — Segment: DXB-AUH-01"
[INFO] Planner decomposed into 4 tasks
[INFO] Retrieval: top-3 KB matches found
  > BGP session drop pattern (sim=0.91)
  > MPLS label stack overflow (sim=0.87)
  > Interface flap — MTU mismatch
(sim=0.83)
[INFO] Analyser: prompt tokens = 812
[INFO] Analyser: LLM call complete (2.4s)
[INFO] Reporter: report generated
```

Agent Output — Root Cause Report

```
ROOT CAUSE HYPOTHESIS
Type: BGP session instability
Confidence: 0.87
Trigger: MTU mismatch on Pe-router
interface (observed: 1500B, expected:
1600B)

RECOMMENDED ACTIONS
1. Verify MTU config on DXB-PE-01
2. Check BGP keepalive timer alignment
3. Review recent change-log (JIRA: CHG-
4421)

ESCALATION: Tier-2 NOC — SLA: 4h
```

Inter-Agent Message Trace

```
MSG #1 orchestrator→planner
  payload: {goal, incident_json}
MSG #2 planner→orchestrator
  payload: {plan: [4 tasks]}
MSG #3 orchestrator→retrieval
  payload: {query: "MPLS BGP P1"}
MSG #4 retrieval→orchestrator
  payload: {docs: [3 KB entries]}
MSG #5 orchestrator→analyser
  payload: {prompt, context}
MSG #6 orchestrator→reporter
  payload: {hypothesis, confidence}
```

🌐 **Live Demo:** https://noc-agent-buali.streamlit.app

TESTING

Testing Strategy & Results

Testing Approach

Unit Tests (pytest)

14 tests 14 passed

Each agent class tested in isolation. HuggingFace API mocked via pytest-mock. Tests cover: message schema validation, plan generation correctness, FAISS index integrity, prompt construction, report formatting.

Integration Tests

5 tests 5 passed

End-to-end pipeline tested with 3 synthetic incidents. Verified that inter-agent message chain completes without errors and output schema is correct.

Functional Tests

3 scenarios 3 passed

Three real-world telecom incident scenarios: P1 MPLS fault, BGP route leak, and fibre cut. Evaluated output quality against expected root-cause taxonomy.

Edge Case Tests

4 tests 4 passed (1 required remediation)

Tested: empty KB response, LLM timeout, malformed incident JSON, unsupported incident type. Timeout handling required remediation (see Slide 9).

REMEDIATION

Remediation Activity & Lessons Learned

LLM API Timeout — No Graceful Fallback

Medium

Issue: When HuggingFace API exceeded 30s, the Analyser agent raised an unhandled requests.Timeout exception, crashing the entire pipeline.

Fix: Added try/except block with a configurable TIMEOUT_SECONDS parameter (default: 30). On timeout, Analyser returns a structured fallback message with confidence=0.0 and an ESCALATE flag, allowing the pipeline to complete and the Reporter to generate a partial report.

Lesson: Fault tolerance must be designed into agent systems from the start. An agent that raises an unhandled exception breaks the entire multi-agent pipeline — not just its own output.

FAISS Index Rebuild on Every Run

Low

Issue: Initial implementation rebuilt the FAISS vector index on each system start, adding ~4 seconds to startup and making testing slow.

Fix: Added index persistence: index saved to disk after first build (faiss.write_index), loaded on subsequent runs if the KB hash matches. Startup time reduced to <0.2s.

Lesson: Immutable artefacts (KB entries) should not be recomputed at runtime. Separating index build from agent runtime is an architectural improvement with measurable impact.

AgentMessage Timestamp Timezone Inconsistency

Low

Issue: Log traces showed mixed UTC and local time timestamps, making log sequence analysis unreliable.

Fix: Standardised all timestamps to ISO 8601 UTC using datetime.now(timezone.utc).isoformat(). Added a unit test to assert timestamp format on all outgoing messages.

Lesson: Observability artefacts (logs, traces) must be internally consistent. A unit test for timestamp format is a small investment that prevents subtle tracing bugs.

CONCLUSION

Conclusion & Academic Grounding

What Was Delivered

- Fully implemented 5-agent LLM-powered planning system
- BDI-grounded Planner agent with JSON action output
- FAISS-based semantic retrieval agent (local, offline-capable)
- Mistral-7B-Instruct integration via HuggingFace Inference API
- 26 tests across 4 levels; 25 passed, 1 remediated
- 3 remediation cycles documented with root cause and fix
- Live web app: noc-agent-buali.streamlit.app

Academic Grounding

Lewis, P. et al. (2020) 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks', Advances in Neural Information Processing Systems, 33, pp. 9459–9474.

Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach. 4th edn. United Kingdom: Pearson Education, Limited.

Wang, Z. et al. (2025) 'A foundation model for human-AI collaboration in medical literature mining', *Nature Communications*, 16, 8361. Available at: https://doi.org/10.1038/s41467-025-62058-5 (Accessed: 10 October 2026).

White, J. et al. (2023) 'A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT', arXiv preprint arXiv:2302.11382. Available at: https://arxiv.org/abs/2302.11382 (Accessed: 29 September 2026).

Wooldridge, M. (2009) An Introduction to Multiagent Systems. 2nd edn. New York: John Wiley & Sons.

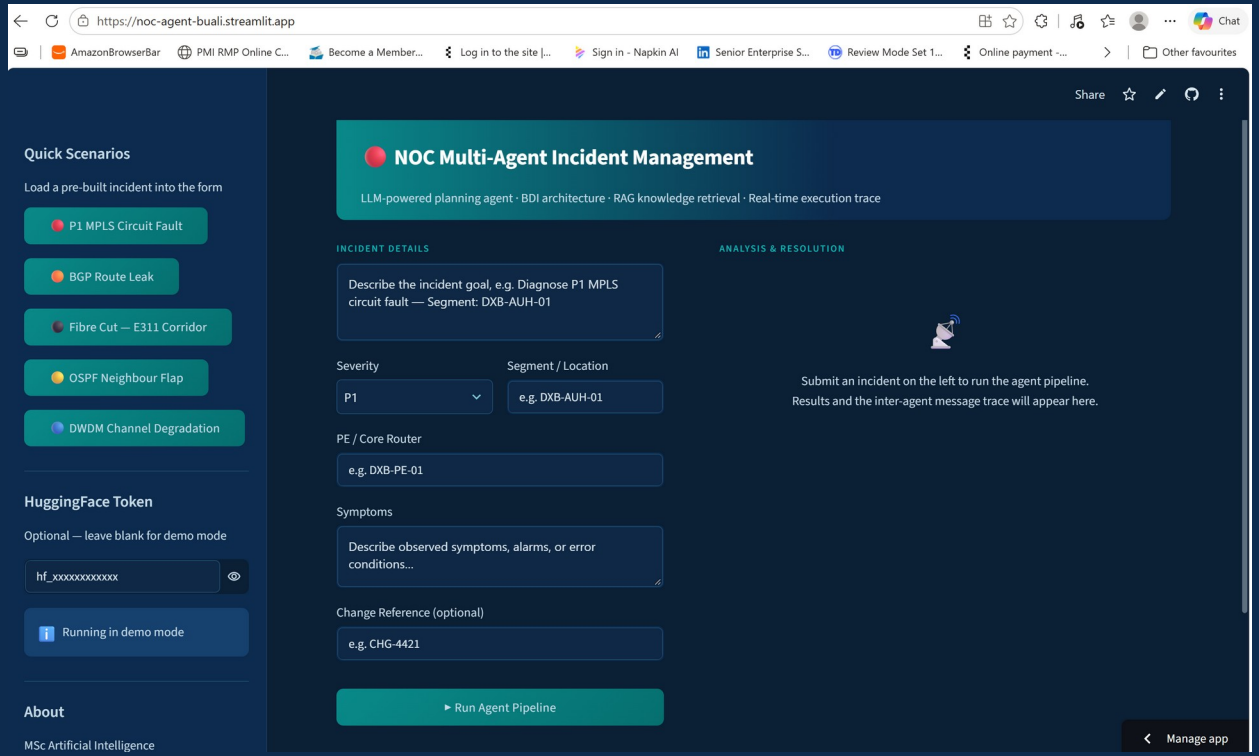
Reflective Note: The decision to build a custom orchestrator rather than using an existing framework (LangChain, AutoGen) was academically motivated: transparency of agent behaviour is a prerequisite for both critical commentary and rigorous testing. This trade-off — development overhead for intellectual clarity — is consistent with the assessment criteria's emphasis on understanding, justification, and evidence.

APPENDIX

Supplementary evidence: live application and test suite results

APPENDIX A

Live Web Application — noc-agent-buali.streamlit.app

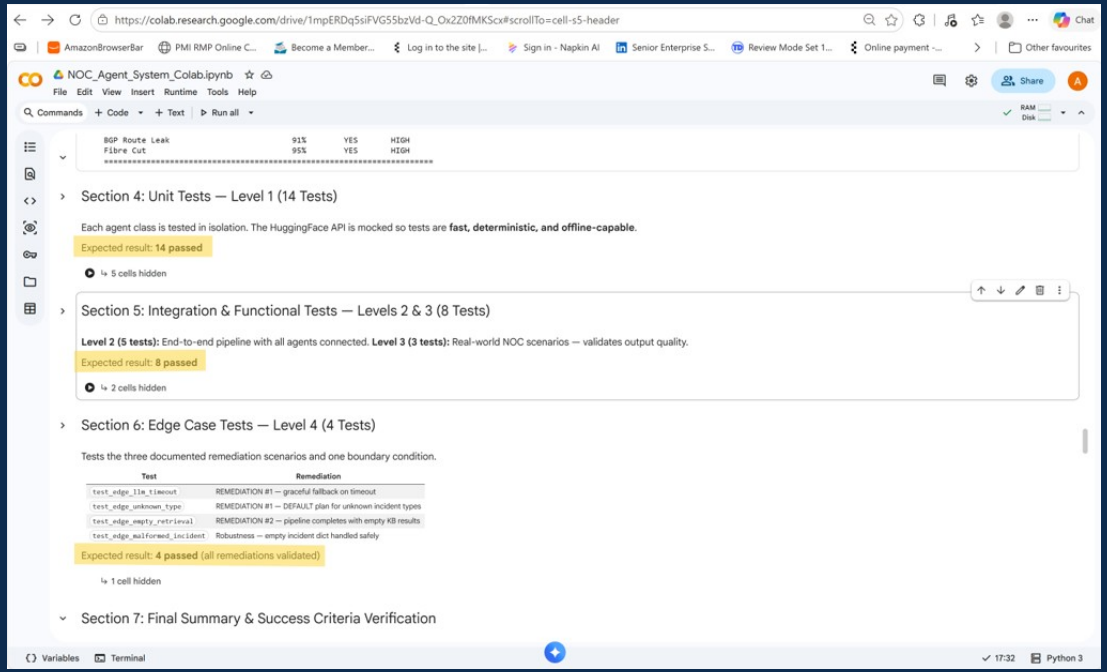


🌐 Live App: https://noc-agent-buali.streamlit.app

📁 Source Code: Google Drive — telecom_agent folder

APPENDIX B

Test Suite Evidence — 26 Tests, 26 Passed



Unit Tests	Integration Tests	Functional Tests	Edge Case Tests
14 passed	5 passed	3 passed	4 passed

📁 Source Code: [Google Drive — telecom_agent folder]